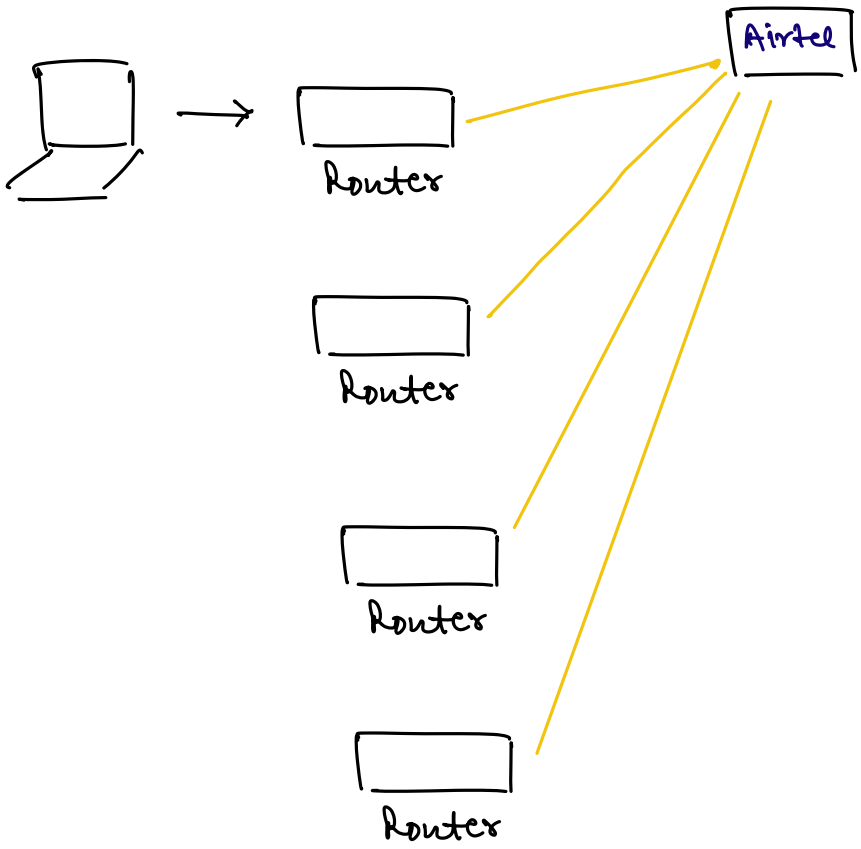
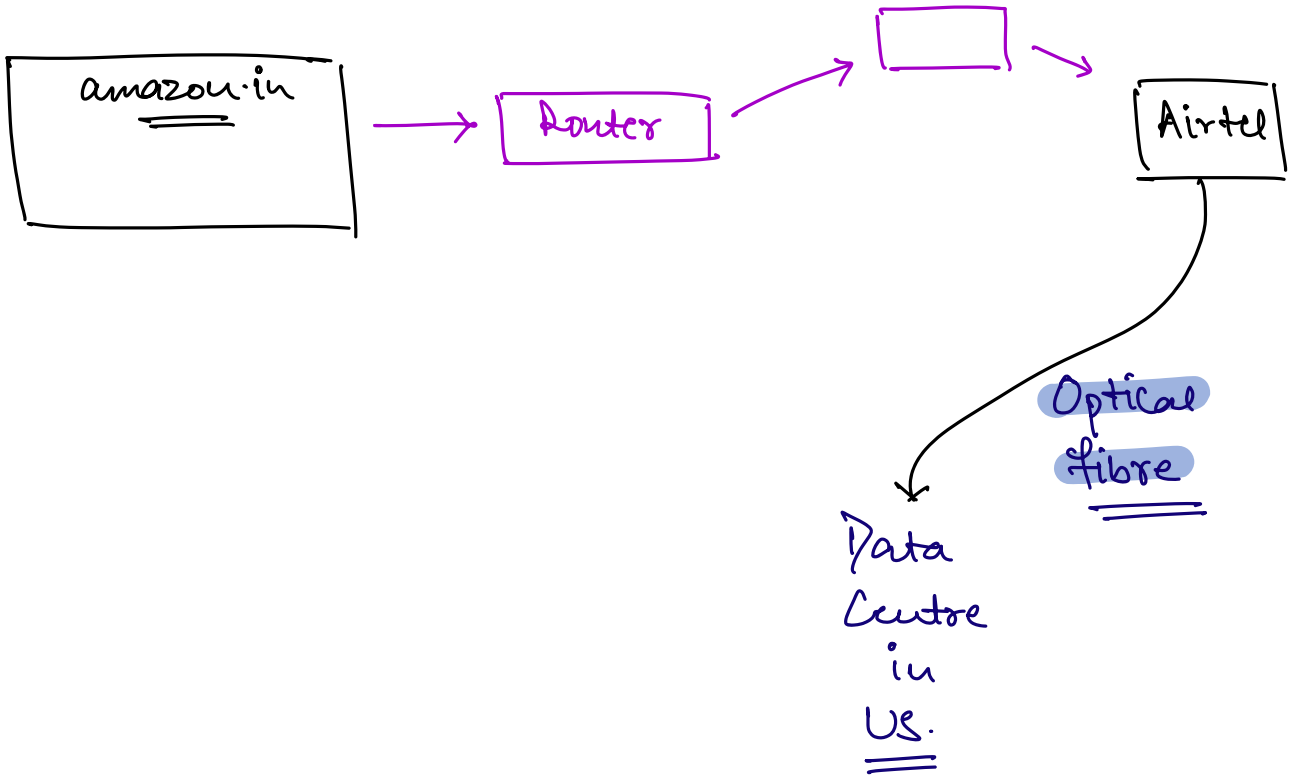
Module-2 : Web Development

- frontend
  - HTML | CSS | JS
  - React JS
- backend
  - Node Js
  - Java + SpringBoot
  - Python + Django | flask | fastAPI

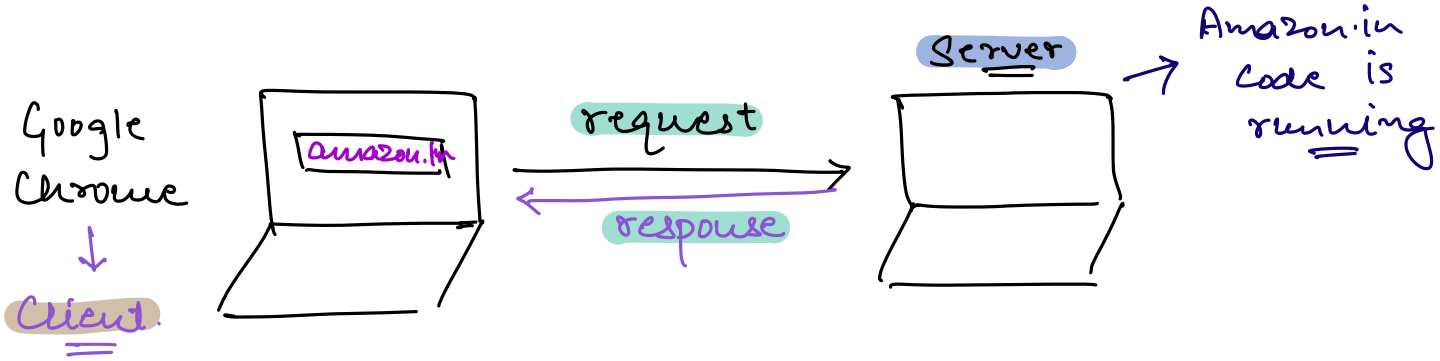
Architecture of Web.

- Internet
- Network





Request - Response Cycle.



Request → Processing → Response.

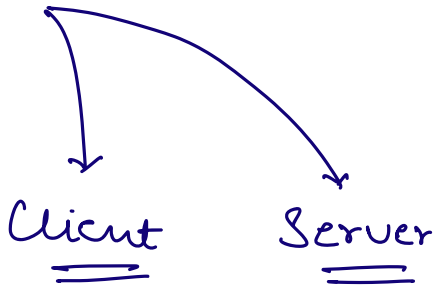
Request - Response Cycle is synchronous.

⇒ Browser waits for the response to come back.

⇒ Every click is round-trip journey.

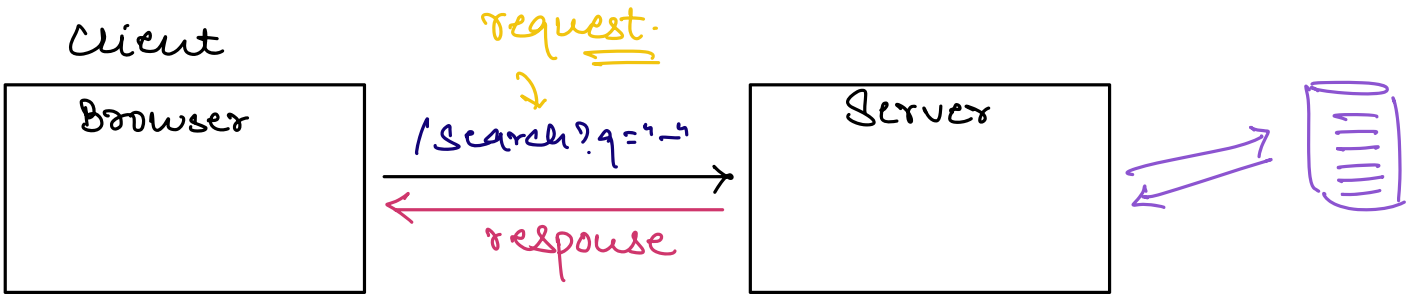
Client - Server Architecture.

⇒ Two computers are involved in every web interaction.



Client : Browser (Chrome | Safari)

↳ Sends a request & displays the response.



⇒ Client connects with DB via Backend Server.

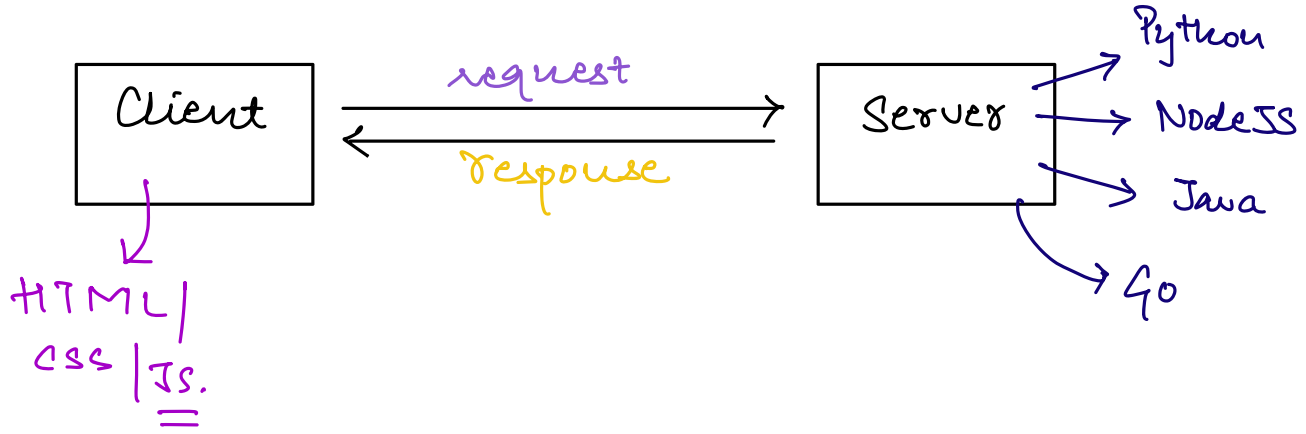
Server : It is a computer somewhere in the Data Center.

It receives the request, processes it, connect to DB (if required) & sends the response back to Client.

JSON.

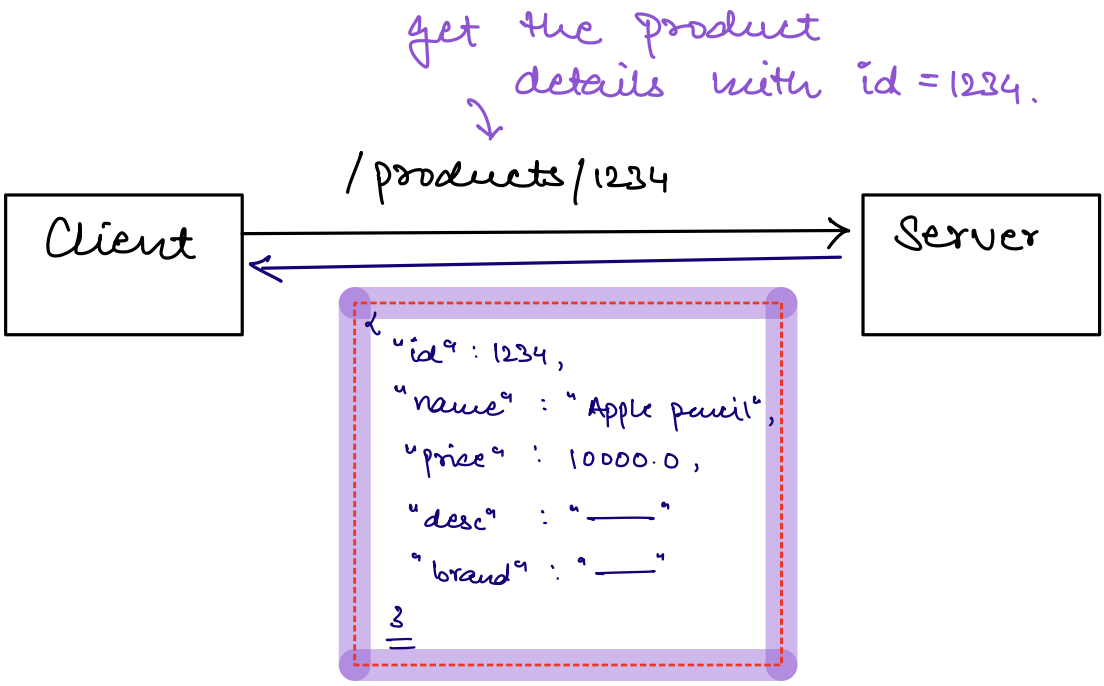
→ JavaScript Object Notation

→ Universal language of data exchange b/w the computers.



JSON ⇒ Every techstack (Python | Java | JS | ...) Supports JSON.

from name, json appears to be an object but in reality it is just text.



Parsing JSON.

⇒

```
jsonString = '{"productId" : 1234, "name" : "Pencil",
               "price" : 10000.0, ..... }'

product = JSON.parse(jsonString)

print(product.name)
```

Cookies.

HTTP → Stateless protocol.

↳ Every http request is completely independent of the previous request.

→ Cookie is a small piece of data (~4KB) that server sends to our browser, and our browser stores this information & uses it in the subsequent requests.

```
// Server sets a cookie in the response header
Set-Cookie: sessionId=abc123; Path=/; HttpOnly

// Browser automatically sends this cookie with future requests
Cookie: sessionId=abc123
```

Cookies. { Session Cookies ⇒ Deleted when we close the browser
Persistent Cookies.
↳ Stored permanently with expiry time.

Sessions

⇒ Sessions are the server-side counter part of the cookies.

⇒ Cookies live in the browser, whereas sessions live on the servers.

```
// Server-side pseudocode
function handleAddToCart(request) {
  const sessionId = request.cookies.sessionId;
  const session = database.getSession(sessionId);

  session.cart.push(request.body.productId);
  database.saveSession(session);

  return { success: true, cart: session.cart };
}
```

CORS.

↳ Cross Origin Resource Sharing

⇒ Browsers have a security rule : A web page can only make requests to its origin (domain)

Same Origin Policy.

CORS is a way for servers to allow the requests coming from different resources as well.

```
// Server response headers for CORS
Access-Control-Allow-Origin: https://smartcart.example.com
Access-Control-Allow-Methods: GET, POST, PUT, DELETE
Access-Control-Allow-Credentials: true
```

If we are building APIs that will be accessed from different domains, we must configure CORS.